INSTITUTO SUPERIOR TECNOLÓGICO QUITO

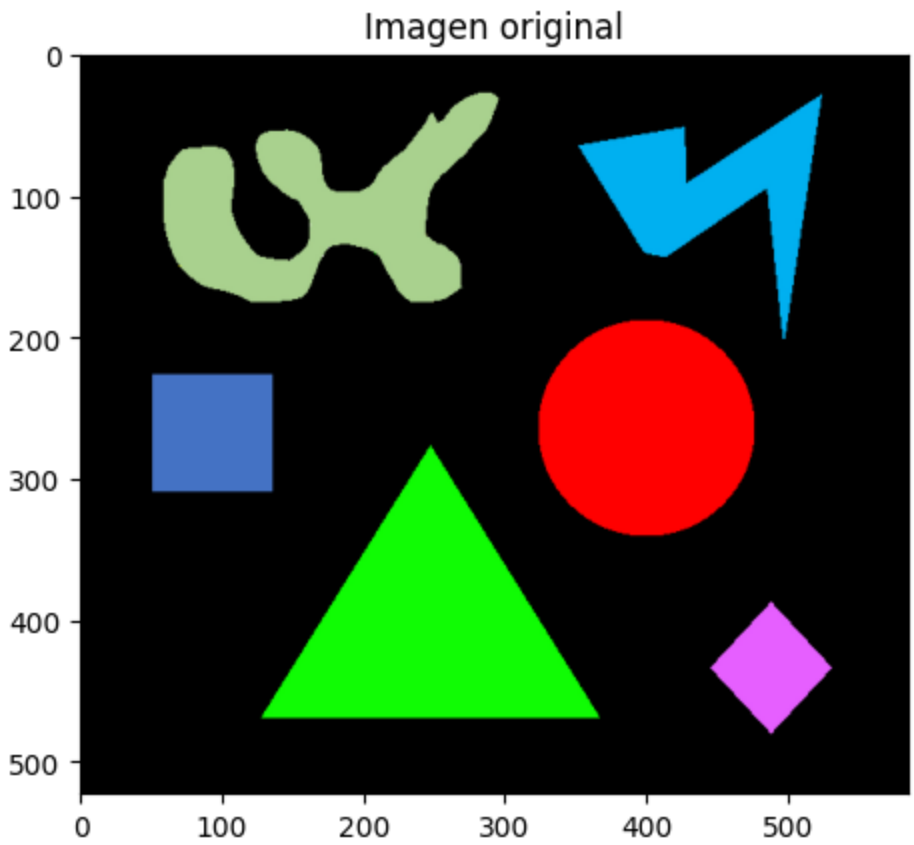
Excelencia en Educación Superior

4.1 Segmentacion de Imagenes

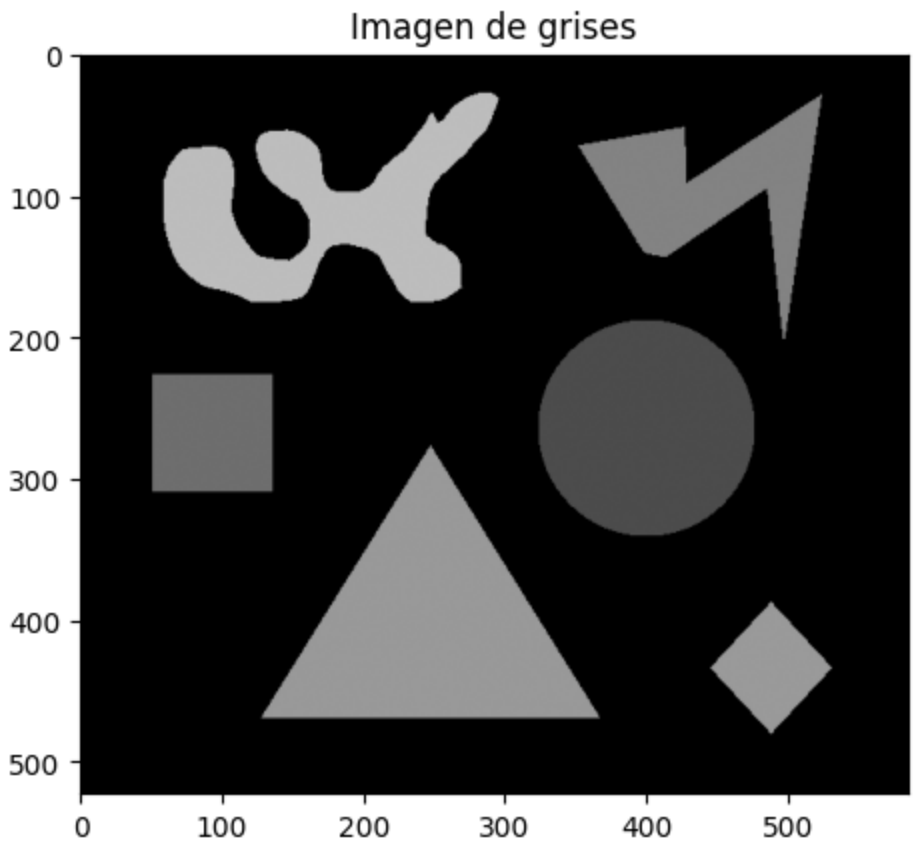
Erick Mejia

SEGMENTACIÓN MEDIANTE THRESHOLDING

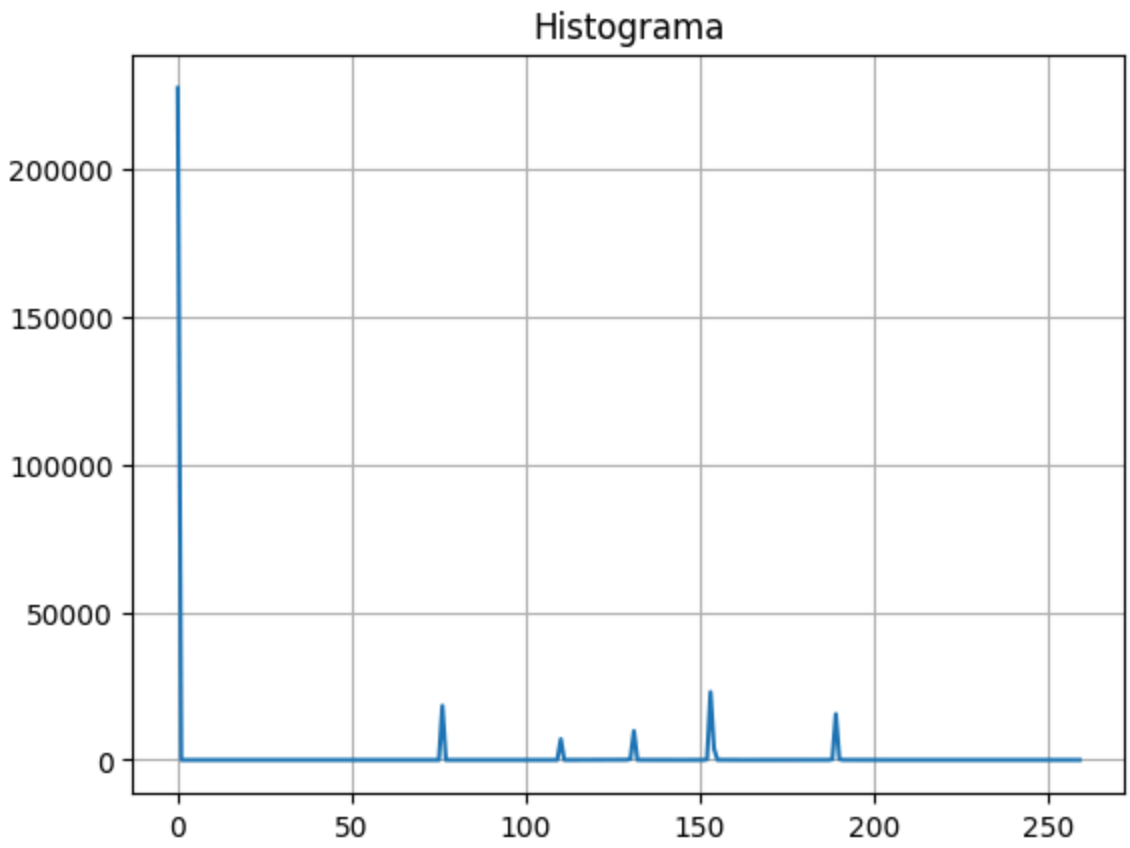
In [40]:
```
import matplotlib.pyplot as plt
import cv2

# Read an image
img = cv2.imread('images/figuras.png')
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
plt.imshow(img_rgb, cmap='gray')
plt.title('Imagen original')
plt.show()
```



In [41]:
```
# Convertimos la imagen a escala de grises
import numpy as np
gray_img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
#gray_img = cv2.imread('figuras.png', 0) # Una forma de leer la imagen directamente

plt.imshow(gray_img, cmap='gray')
plt.title('Imagen de grises')
plt.show()
```



In [42]:
```
# Calculamos el histograma
hist = cv2.calcHist([gray_img], [0], None, [260], [0, 260]) # [0] son los canales,

plt.plot(hist)
plt.grid()
plt.title('Histograma')
plt.show()

# Otra forma de mostrar el histograma (solo visualización)
#plt.hist(gray_img.ravel(), bins=50)
#plt.grid(True)
#plt.show()
```



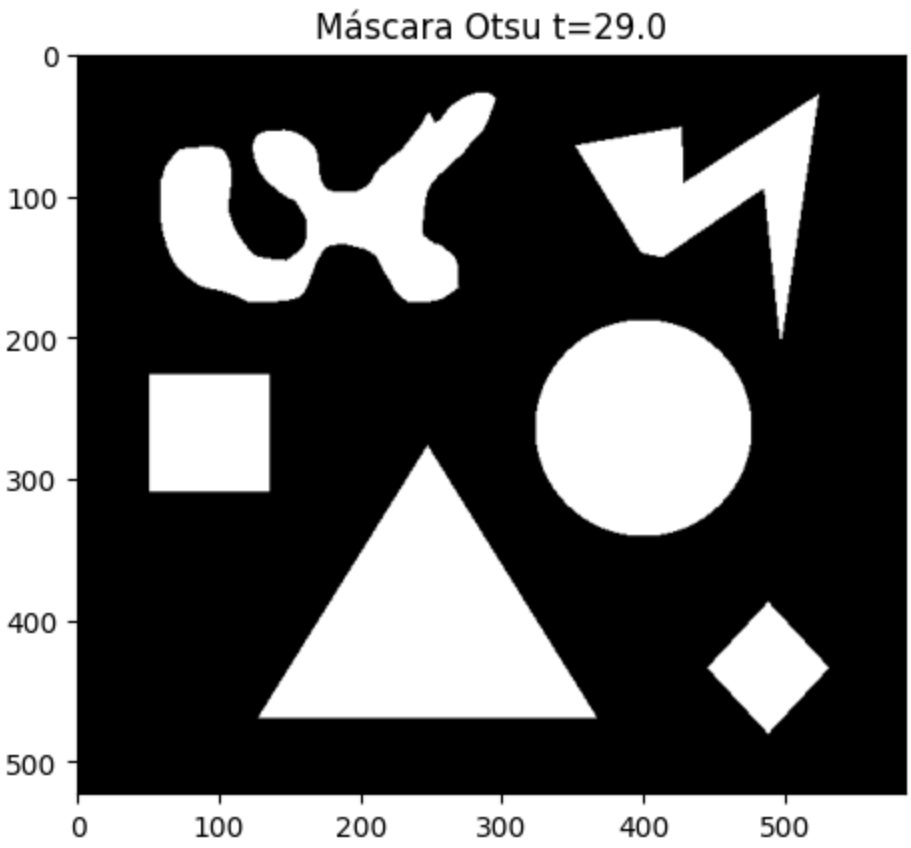
In [43]:
```
# Fijamos el umbral en base al histograma anterior
t = 50

# Extreaemos la máscara binaria
maxim = 255
_, mask = cv2.threshold(gray_img, t, maxim, cv2.THRESH_BINARY)

# Otra formas de extraer la máscara
# mask = gray_img.copy()
# mask = mask>t

# Visualizamos para corroborar
plt.imshow(mask, cmap='gray')
plt.title('Máscara t=' + str(t))
plt.show()

print(np.unique(mask)) # Atent@s a los formatos (bool, uint8, etc.)
```

```
[  0 255]
```

In [44]:

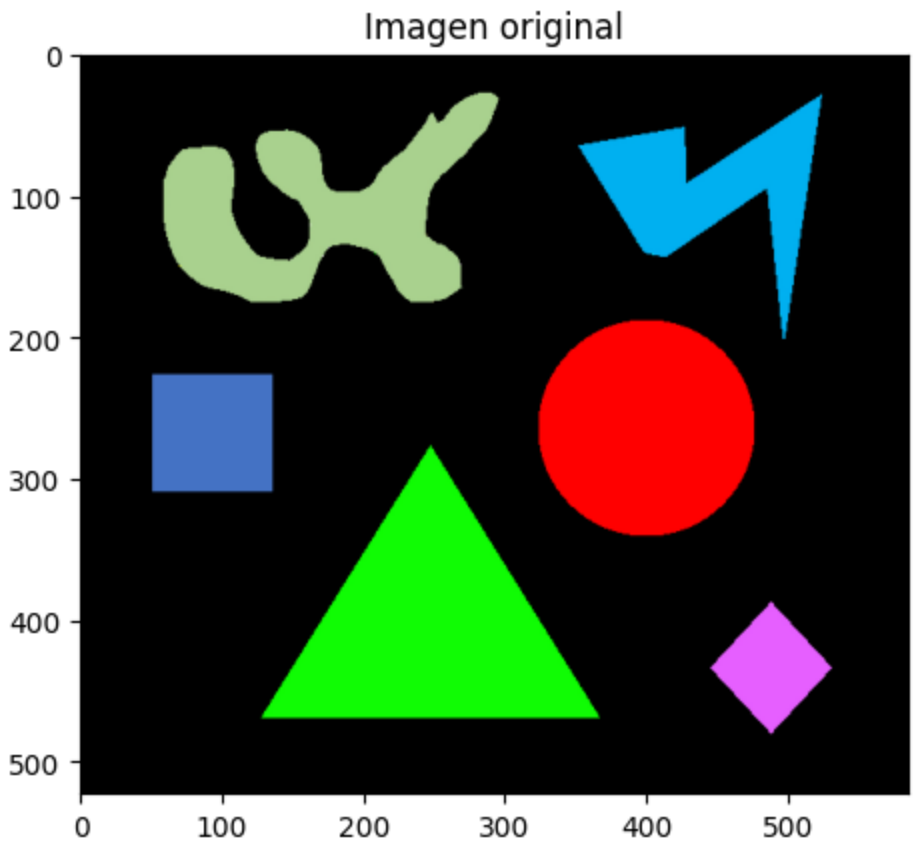
```python
# Fijamos el umbral con el método de OTSU
t, mask = cv2.threshold(gray_img,0,1,cv2.THRESH_OTSU) # 0 es por defecto y 1 es el
print(np.unique(mask))

# Visualizamos para corroborar que se obtiene el mismo resultado
plt.imshow(mask, cmap='gray')
plt.title('Máscara Otsu t=' + str(t))
plt.show()
```

```
[0 1]
```

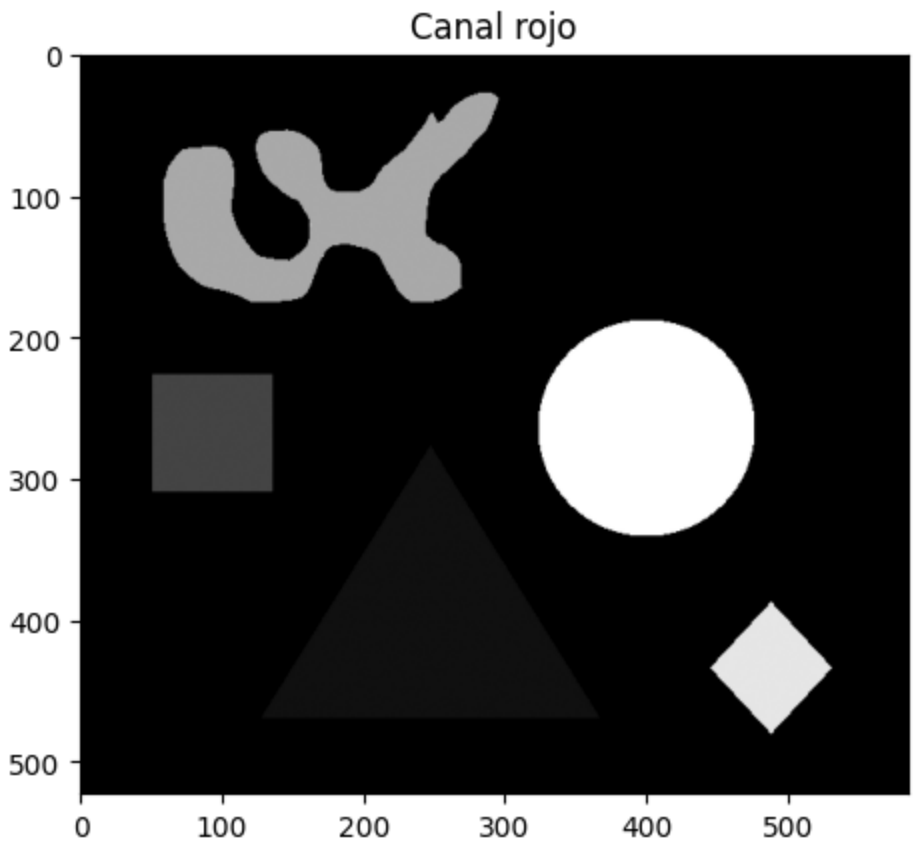


SEGMENTACIÓN MEDIANTE CONVERSIONES DE COLOR

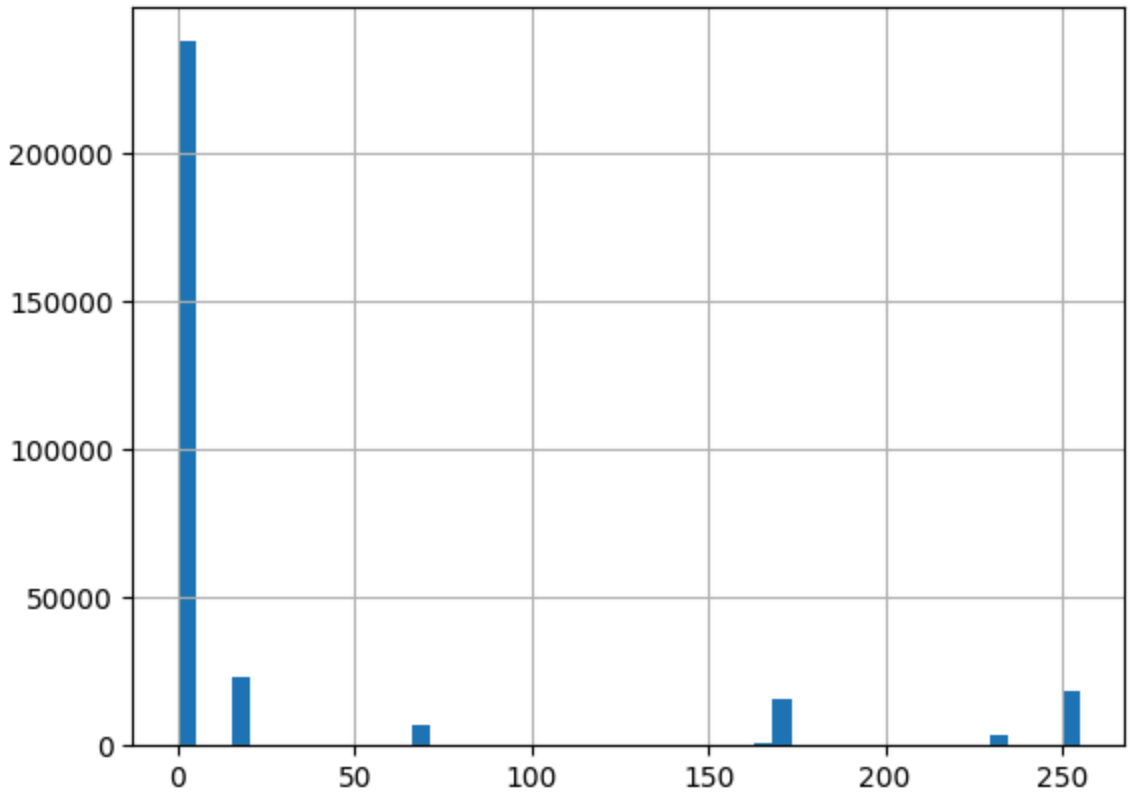
In [45]:
```
# Imaginamos que queremos segmentar únicamente el círculo
img = cv2.imread('images/figuras.png')
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

plt.imshow(img_rgb, cmap='gray')
plt.title('Imagen original')
plt.show()
```



In [46]:
```
# Obtenemos el canal rojo
R = img[:,:,2]

plt.imshow(R, cmap='gray')
plt.title('Canal rojo')
plt.show()
```

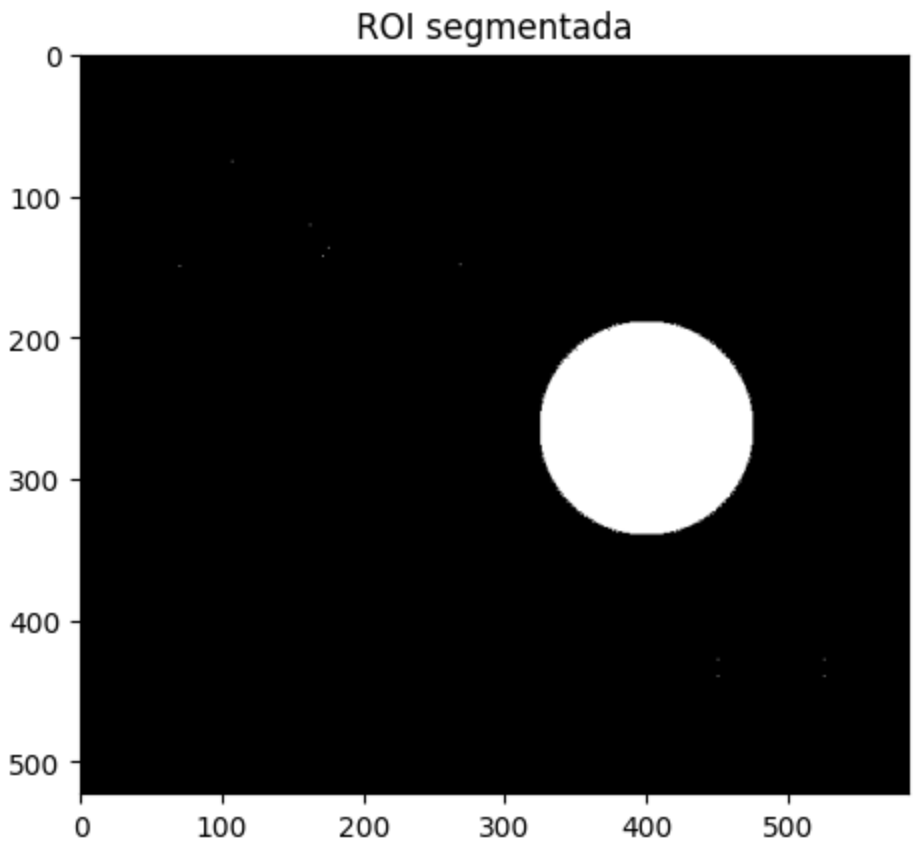


In [47]:
```
# Observamos el histograma
plt.hist(R.ravel(), bins=50)
plt.grid(True)
plt.show()
print(np.max(R))
```



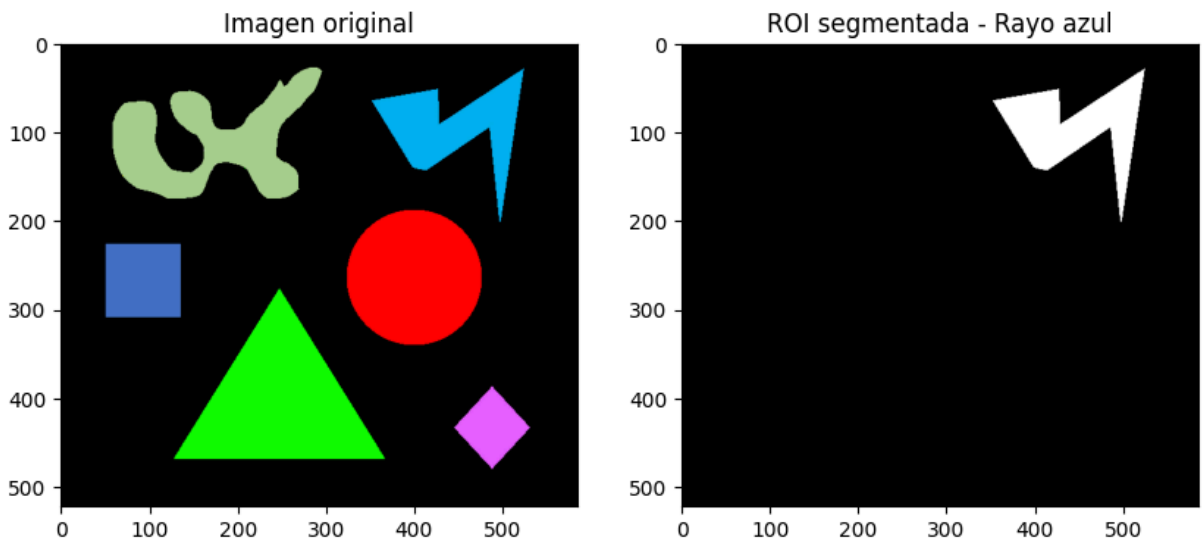

```
255
```

In [48]:
```
# Fijamos el umbral y extraemos la máscara
t=np.max(R)
mask = R.copy()
mask[mask<t] = 0

plt.imshow(mask, cmap='gray')
plt.title('ROI segmentada')
plt.show()
```



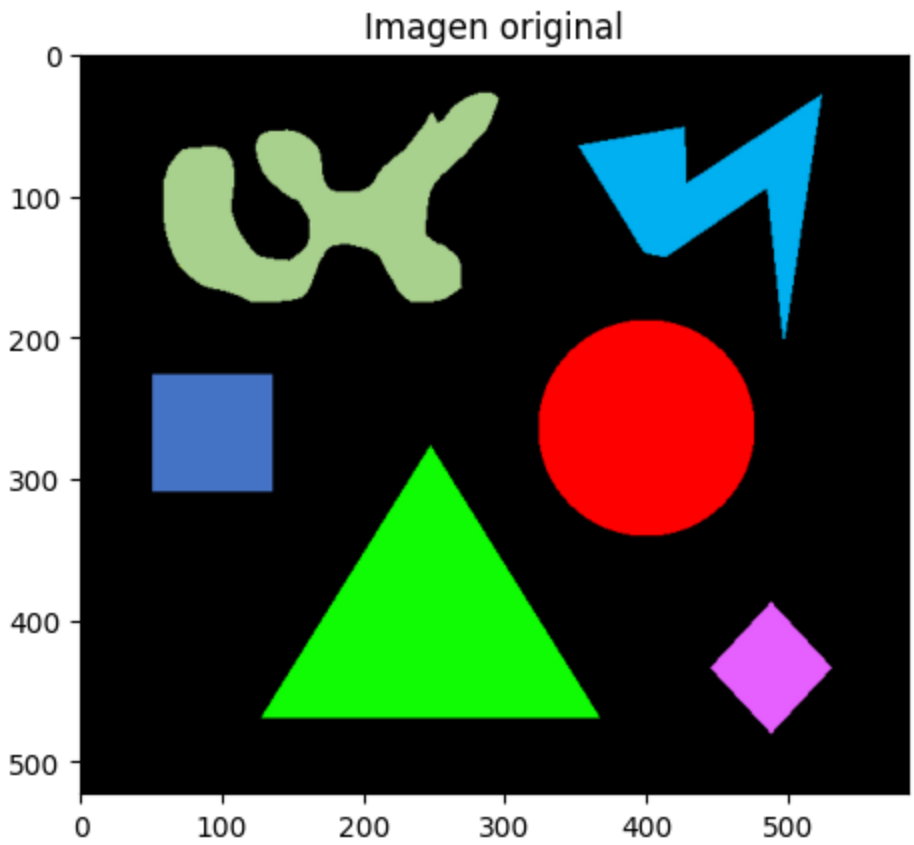
In [49]:
```
# Ejercicio individual
# Obtener una máscara con la imagen del rayo azul mediante segmentación por espacio

# Función para extraer los colores secundarios
def convert_to_CMYK(rgb_p):
    with np.errstate(invalid='ignore', divide='ignore'):
        K = 1 - np.max(rgb_p, axis=2)
        C = (1 - rgb_p[:, :, 0] - K) / (1 - K)
        M = (1 - rgb_p[:, :, 1] - K) / (1 - K)
        Y = (1 - rgb_p[:, :, 2] - K) / (1 - K)

    # Reemplazar NaN e Inf por 0 antes de convertir
    C = np.nan_to_num(C, nan=0.0, posinf=0.0, neginf=0.0)
    M = np.nan_to_num(M, nan=0.0, posinf=0.0, neginf=0.0)
    Y = np.nan_to_num(Y, nan=0.0, posinf=0.0, neginf=0.0)
    K = np.nan_to_num(K, nan=0.0, posinf=0.0, neginf=0.0)

    CMYK = (np.dstack((C, M, Y, K)) * 255).astype('uint8')
    return CMYK

# Leer la imagen
img = cv2.imread('images/figuras.png')
rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

# Convertir a HSV
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)

# Rango azul (cuadrado)
lower_blue = np.array([100, 50, 50])
```

```
upper_blue = np.array([130, 255, 255])

# Rango cian (rayo)
lower_cyan = np.array([85, 50, 50])
upper_cyan = np.array([100, 255, 255])

# Crear máscaras individuales
mascara_blue = cv2.inRange(hsv, lower_blue, upper_blue)
mascara_cyan = cv2.inRange(hsv, lower_cyan, upper_cyan)

# Máscara solo del rayo (cian)
mascara_rayo = mascara_cyan

# Visualizar
fig, axes = plt.subplots(1, 2, figsize=(10, 5))
axes[0].imshow(rgb)
axes[0].set_title('Imagen original')
axes[1].imshow(mascara_rayo, cmap='gray')
axes[1].set_title('ROI segmentada - Rayo azul')
plt.show()
```

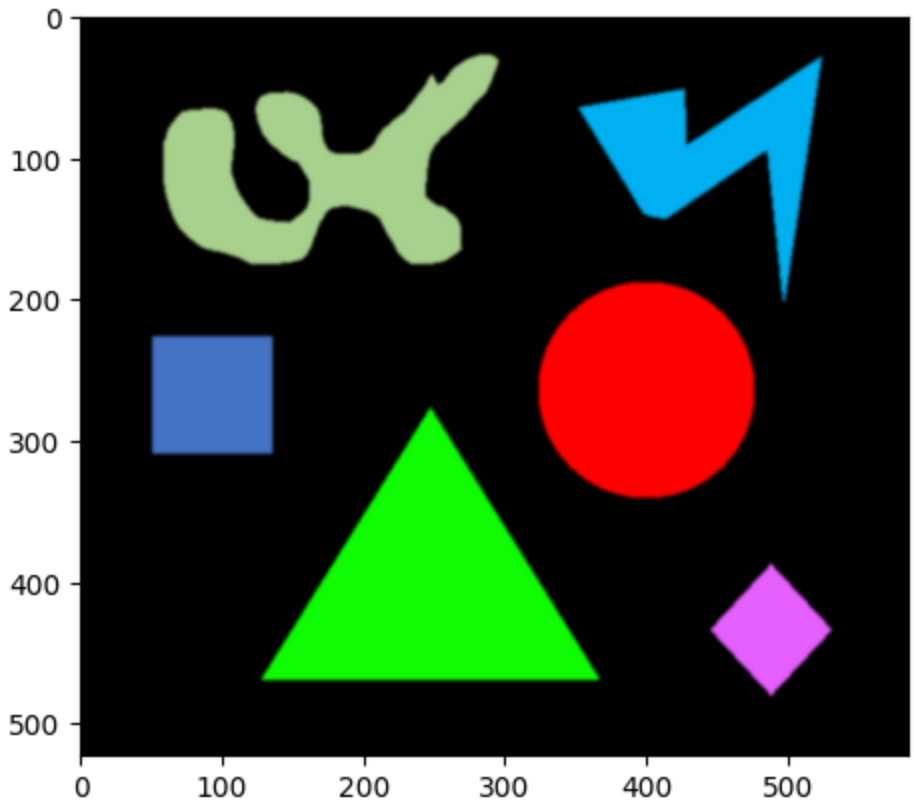


SEGMENTACIÓN SEMÁNTICA CON DETECCIÓN DE BORDES

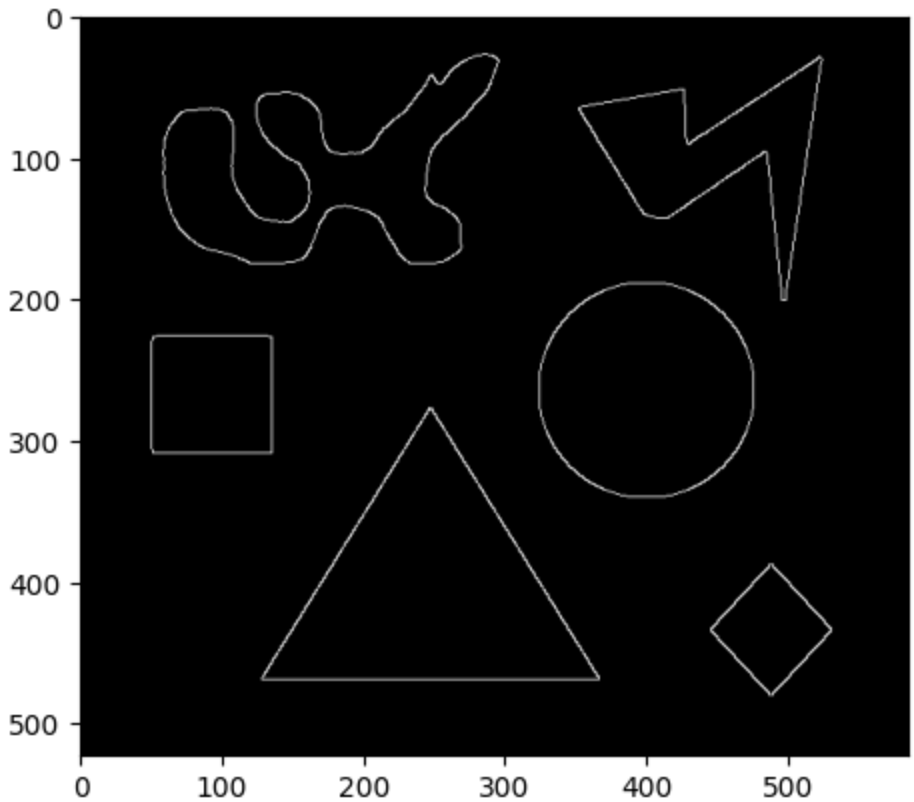
In [50]:
```
# SEGMENTACIÓN SEMÁNTICA CON DETECCIÓN DE BORDES
img = cv2.imread('images/figuras.png')
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

plt.imshow(img_rgb)
plt.title('Imagen original')
plt.show()
```



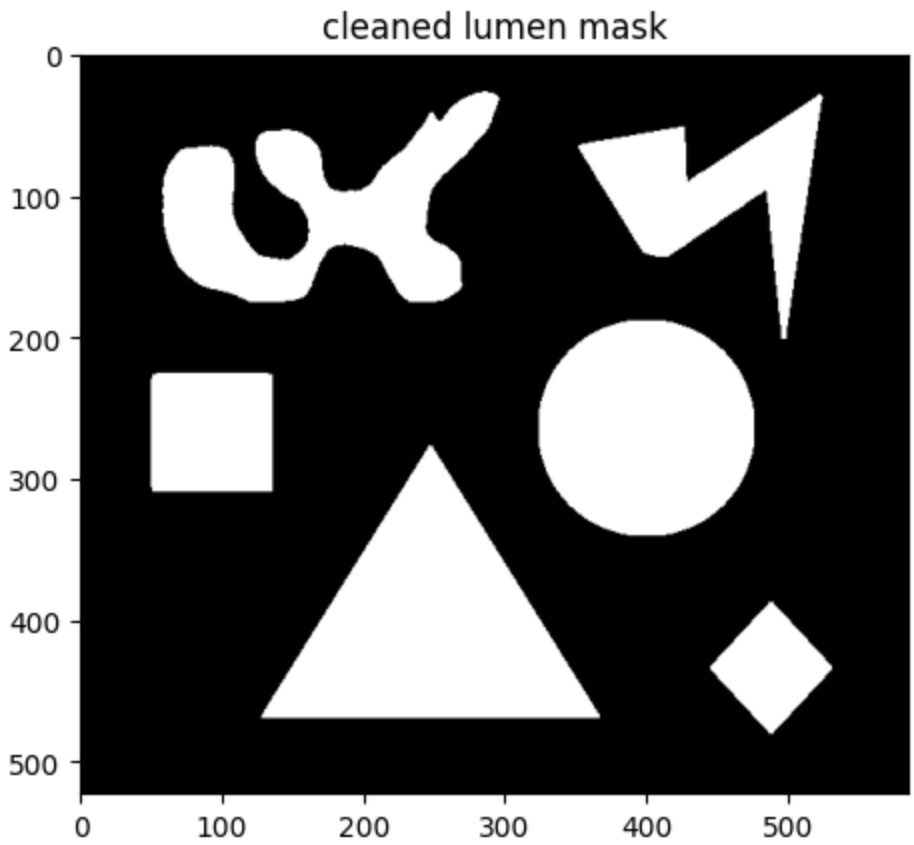
In [51]:
```
# Aplicamos un filtro gaussiano para emborronar las altas frecuencias
img_gaus = cv2.GaussianBlur(img_rgb, (5,5), 0) # (5x5) es el tamaño del filtro y 0
plt.imshow(img_gaus, cmap='gray')
plt.show()
```



In [52]:
```
# Detección de bordes de Canny
canny = cv2.Canny(img_gaus, 255/3, 255) # minValue and maxValue. Canny recomienda q
kernel = np.ones((2,2), 'uint8')
#canny = cv2.dilate(canny, kernel, iterations=2)
#canny = cv2.erode(canny, kernel, iterations=2)

plt.imshow(canny, cmap='gray')
plt.show()
```

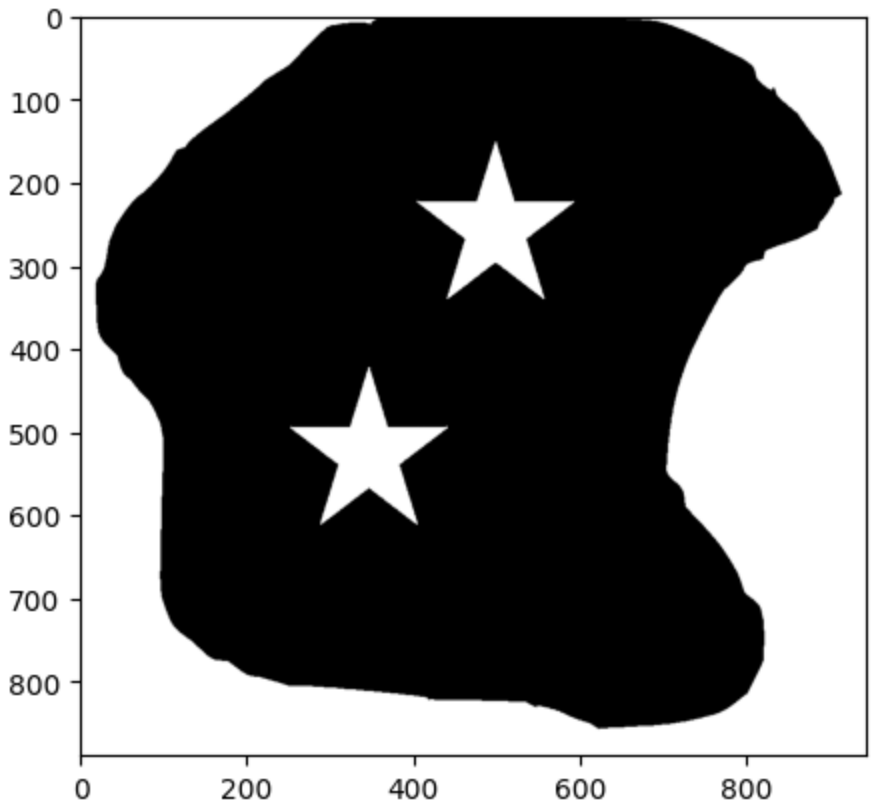


In [53]:
```
# Rellenar objetos (solo rellena si no están en contacto con el límite de la imagen
from scipy.ndimage import binary_fill_holes as bfh

bordes = canny.copy()
filled_bordes = bfh(bordes)

plt.imshow(filled_bordes.astype('uint8'), cmap='gray')
plt.title('cleaned lumen mask')
plt.show()
```

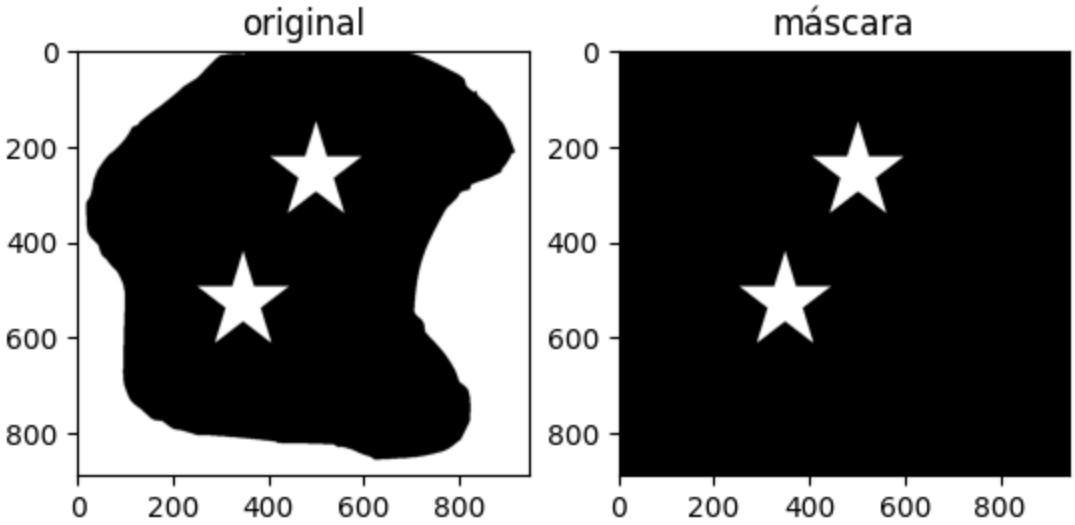


SEGMENTACIÓN POR CRECIMIENTO DE REGIONES (region growing)

In [54]:
```
# Cargar imagen
from skimage import io
import cv2
import numpy as np
import matplotlib.pyplot as plt
img = cv2.imread('images/estrellas.png', 0)

plt.imshow(img, cmap='gray')
plt.show()
```



In [55]:
```
# Crear una copia de la imagen
mask = img.copy()

# Definir una imagen negra de referencia
h, w = img.shape
ref = np.zeros((h+2, w+2), np.uint8)

# Fijar la semilla. En este caso en la esquina superior izquierda (posición 0,0) pa
cv2.floodFill(mask, ref, (0,0), 0); # Como nos interesa rellenar de negro, utilizam

fig, ax = plt.subplots(1,2)
ax[0].imshow(img, cmap='gray'), ax[0].set_title('original')
ax[1].imshow(mask, cmap='gray'), ax[1].set_title('máscara')
plt.show()
```



5. Github

https://github.com/Erick-305/machine-learning.git